

Juego de Fichas

Main

```
import java.util.Scanner;
public class JuegoDeFichas {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        Casilla casilla = new Casilla();
        int indice;
        boolean start = true;
        String[][] tablero = new String[casilla.getFila()][casilla.getColumna()];
        for(int i = 0; i<5; i++){
            System.out.println("\n");
            for(int j = 0; j<tablero[i].length;j++){
                tablero[i][j]= "0";
                System.out.print("\t"+tablero[i][j]);
            }
        }
        while(start){
            System.out.println("\n\t\tJUEGO DE FICHAS");
            System.out.println("1. Normas\n2. Juego básico\n3. Juego Avanzado\n4. Exit");
            indice = in.nextInt();
            switch(indice){
                case 1:
                    normas();
                    break;
                case 2:
                    juegoBasico(tablero);
                    break;
                case 3:
                    juegoAvanzado(tablero);
                    break;
                case 4:
                    start = false;
                    break;
                default:
                    System.out.println("Introduce una opción correcta");
                    System.out.println("\t\tJUEGO DE FICHAS");
                    System.out.println("1. Nomas\n2. Juego básico\n3. Juego Avanzado\n4. Exit");
            }
        }
    }
}
```

```
//Una breve explicación de como funciona el juego
public static void normas(){
    System.out.println("\t\tJuego Básico");
    System.out.println("""
        Para empezar el juego se pide por teclado de qu\u00e9 tipo ser\u00e1 la
        ficha blanca y qu\u00e9 posici\u00f3n tendr\u00e1.
        Seguidamente se pedir\u00e1 de qu\u00e9 tipo ser\u00e1 la ficha negra y
        su posici\u00f3n, que no puede ser la casilla inicial de la ficha blanca.
        Empieza la ficha blanca y se mueve para intentar capturar la ficha negra.
        A continuaci\u00f3n se mueve la ficha negra para intentar capturar la ficha
        blanca. Y así sucesivamente... Gana la ficha que captura a la otra.""");
    System.out.println("\n\t\tJuego Avanzado");
    System.out.println("""
        El Alfil tendr\u00e1 vidas extras que se indicaran al principio de la partida
        La Torre tendr\u00e1 un turno extra que se indicaran al principio del
        juego""");
}
```

```
public static void juegoBasico(String[][]tablero){
    Scanner in = new Scanner(System.in);
    Ficha fichaBlanca = crearFichaBlanca();
    boolean posicionInicialNoRepetida = false;
    Ficha fichaNegra = null;
    while(!posicionInicialNoRepetida){
        fichaNegra = crearFichaNegra();
        if((fichaBlanca.getPosicionX()== fichaNegra.getPosicionX()) &&
(fichaBlanca.getPosicionY()== fichaNegra.getPosicionY())){
            System.out.println("La ficha negra no puede empezar en la misma posicion que la
            blanca\nCrea de nuevo la ficha");
        }else{
            posicionInicialNoRepetida = true;
        }
    }
    boolean hayGanador = false;
    tablero[fichaBlanca.getPosicionX()][fichaBlanca.getPosicionY()] =
fichaBlanca.getNombre()+fichaBlanca.getColor();
    tablero[fichaNegra.getPosicionX()][fichaNegra.getPosicionY()]=
fichaNegra.getNombre()+fichaNegra.getColor();
    while(!hayGanador){
        String movimiento;
        boolean movimientoCorrecto = false;
        for(int i = 0; i<5; i++){
            System.out.println("\n");
            for(int j = 0; j<tablero[i].length;j++){
                System.out.print("\t"+tablero[i][j]);
            }
        }
    }
}
```

```

tablero[fichaBlanca.getPosicionX()][fichaBlanca.getPosicionY()] = "0";
while(!movimientoCorrecto){
    int movimientoCorrectoX,movimientoCorrectoY;
    System.out.println("\nMueve la ficha blanca\nElige el eje X");
    movimiento = in.nextLine();
    movimientoCorrectoX=(valorCorrecto(movimiento));
    System.out.println("Elige el eje J");
    movimiento = in.nextLine();
    movimientoCorrectoY=(valorCorrecto(movimiento));
    Casilla destino = new Casilla(movimientoCorrectoX,movimientoCorrectoY);
    if(fichaBlanca.moverA(destino)){
        fichaBlanca.setPosicionX(movimientoCorrectoX);
        fichaBlanca.setPosicionY(movimientoCorrectoY);
        movimientoCorrecto = true;
    }else{
        System.out.println("Movimiento incorrecto");
    }
}
movimientoCorrecto = false;
tablero[fichaBlanca.getPosicionX()][fichaBlanca.getPosicionY()] =
fichaBlanca.getNombre()+fichaBlanca.getColor();
if(fichaBlanca.getPosicionX()== fichaNegra.getPosicionX() &&
fichaBlanca.getPosicionY() == fichaNegra.getPosicionY()){
    System.out.println("\nLas Blancas han Ganado");
    for(int i = 0; i<5; i++){
        System.out.println("\n");
        for(int j = 0; j<tablero[i].length;j++){
            System.out.print("\t"+tablero[i][j]);
        }
    }
    hayGanador = true;
}else{
    for(int i = 0; i<5; i++){
        System.out.println("\n");
        for(int j = 0; j<tablero[i].length;j++){
            System.out.print("\t"+tablero[i][j]);
        }
    }
    tablero[fichaNegra.getPosicionX()][fichaNegra.getPosicionY()] = "0";
    while(!movimientoCorrecto){
        int movimientoCorrectoX,movimientoCorrectoY;
        System.out.println("\nMueve la ficha negra\nElige el eje X");
        movimiento = in.nextLine();
        movimientoCorrectoX=(valorCorrecto(movimiento));
        System.out.println("Elige el eje J");
        movimiento = in.nextLine();
        movimientoCorrectoY=(valorCorrecto(movimiento));
        Casilla destino = new Casilla(movimientoCorrectoX,movimientoCorrectoY);

```

```

        if(fichaNegra.moverA(destino)){
            fichaNegra.setPosicionX(movimientoCorrectoX);
            fichaNegra.setPosicionY(movimientoCorrectoY);
            movimientoCorrecto = true;
        }else{
            System.out.println("Movimiento incorrecto");
        }
    }
    tablero[fichaNegra.getPosicionX()][fichaNegra.getPosicionY()] =
fichaNegra.getNombre()+fichaNegra.getColor();
    if(fichaBlanca.getPosicionX()== fichaNegra.getPosicionX() &&
fichaBlanca.getPosicionY() == fichaNegra.getPosicionY()){
        System.out.println("\nLas Negras han Ganado");
        for(int i = 0; i<5; i++){
            System.out.println("\n");
            for(int j = 0; j<tablero[i].length;j++){
                System.out.print("\t"+tablero[i][j]);
            }
        }
        hayGanador = true;
    }
}
}
}
}
}

```

```

//Comprovamos que los movimientos dentro del juego no sean erroneos
public static int valorCorrecto(String movimiento){
    Scanner in = new Scanner(System.in);
    boolean datoCorrecto = false;
    int posicionCorrecta = 0;
    while(!datoCorrecto){
        try{
            posicionCorrecta = Integer.parseInt(movimiento);
            if(posicionCorrecta >= 0 && posicionCorrecta <= 4){
                datoCorrecto = true;
            }else{
                System.out.println("Debes de introducir un número entre 0 y 4");
                movimiento = in.nextLine();
            }
        }catch(NumberFormatException e){
            System.out.println("Debes de introducir un valor numérico sin decimales");
            System.out.println("posicion Y(de 0 a 4):");
            movimiento = in.nextLine();
        }
    }
    return posicionCorrecta;
}
}

```

```

//Metodo para crear la ficha BLanca en el juego básico
public static Ficha crearFichaBlanca(){
    Scanner in = new Scanner(System.in);
    Ficha alfil1 = new Alfil();
    Ficha caballo1 = new Caballo();
    Ficha torre1 = new Torre();
    boolean datoCorrecto = false;
    String eleccionFicha,posicionX,posicionY;
    int datoCorrectoFicha = 0;
    int posicionXCorrecta = -1;
    int posicionYCorrecta = -1;
    while(!datoCorrecto){
        System.out.println("Elige la primera ficha Blanca:\n1: Alfil\n2. Caballo\n3. Torre");
        eleccionFicha = in.nextLine();
        try{
            datoCorrectoFicha = Integer.parseInt(eleccionFicha);
            if(datoCorrectoFicha >=1 && datoCorrectoFicha<=3){
                datoCorrecto = true;
            }else{
                System.out.println("Debes de introducir un número entre 1 y 3");
            }
        }catch(NumberFormatException e){
            System.out.println("Debes de introducir un valor numérico sin decimales");
        }
    }
    datoCorrecto = false;
    System.out.println("Elige una posicion de inicio X e Y");
    while(!datoCorrecto){
        System.out.println("Posicion X (de 0 a 4):");
        posicionX = in.nextLine();
        try{
            posicionXCorrecta = Integer.parseInt(posicionX);
            if(posicionXCorrecta >= 0 && posicionXCorrecta <= 4){
                datoCorrecto = true;
            }else{
                System.out.println("Debes de introducir un número entre 0 y 4");
            }
        }catch(NumberFormatException e){
            System.out.println("Debes de introducir un valor numérico sin decimales");
        }
    }
    datoCorrecto = false;
    while(!datoCorrecto){
        System.out.println("posicion Y(de 0 a 4):");
        posicionY = in.nextLine();
        try{
            posicionYCorrecta = Integer.parseInt(posicionY);
            if(posicionYCorrecta >= 0 && posicionYCorrecta <= 4){

```

```

        datoCorrecto = true;
    }else{
        System.out.println("Debes de introducir un número entre 0 y 4");
    }
}catch(NumberFormatException e){
    System.out.println("Debes de introducir un valor numérico sin decimales");
}
}
}

switch (datoCorrectoFicha) {
    case 1:
        alfil1.setColor("b");
        alfil1.setPosicionX(posicionXCorrecta);
        alfil1.setPosicionY(posicionYCorrecta);
        return alfil1;
    case 2:
        caballo1.setColor("b");
        caballo1.setPosicionX(posicionXCorrecta);
        caballo1.setPosicionY(posicionYCorrecta);
        return caballo1;
    case 3:
        torre1.setColor("b");
        torre1.setPosicionX(posicionXCorrecta);
        torre1.setPosicionY(posicionYCorrecta);
        return torre1;
    default:
        break;
}
return null;
}

```

//Metodo para crear la ficha Negra en el juego básico

```

public static Ficha crearFichaNegra(){
    Scanner in = new Scanner(System.in);
    Ficha alfil2 = new Alfil();
    Ficha caballo2 = new Caballo();
    Ficha torre2 = new Torre();
    boolean datoCorrecto = false;
    String eleccionFicha,posicionX,posicionY;
    int datoCorrectoFicha = 0;
    int posicionXCorrecta = -1;
    int posicionYCorrecta = -1;
    while(!datoCorrecto){
        System.out.println("Elige la primera ficha Negra:\n1: Alfil\n2. Caballo\n3. Torre");
        eleccionFicha = in.nextLine();
        try{
            datoCorrectoFicha = Integer.parseInt(eleccionFicha);
            if(datoCorrectoFicha >=1 && datoCorrectoFicha<=3){
                datoCorrecto = true;
            }
        }
    }
}

```

```

        }else{
            System.out.println("Debes de introducir un número entre 1 y 3");
        }
    }catch(NumberFormatException e){
        System.out.println("Debes de introducir un valor numérico sin decimales");
    }
}
datoCorrecto = false;
System.out.println("Elije una posicion de inicio X e Y");
while(!datoCorrecto){
    System.out.println("Posicion X (de 0 a 4):");
    posicionX = in.nextLine();
    try{
        posicionXCorrecta = Integer.parseInt(posicionX);
        if(posicionXCorrecta >= 0 && posicionXCorrecta <= 4){
            datoCorrecto = true;
        }else{
            System.out.println("Debes de introducir un número entre 0 y 4");
        }
    }catch(NumberFormatException e){
        System.out.println("Debes de introducir un valor numérico sin decimales");
    }
}
datoCorrecto = false;
while(!datoCorrecto){
    System.out.println("posicion Y(de 0 a 4):");
    posicionY = in.nextLine();
    try{
        posicionYCorrecta = Integer.parseInt(posicionY);
        if(posicionYCorrecta >= 0 && posicionYCorrecta <= 4){
            datoCorrecto = true;
        }else{
            System.out.println("Debes de introducir un número entre 0 y 4");
        }
    }catch(NumberFormatException e){
        System.out.println("Debes de introducir un valor numérico sin decimales");
    }
}
switch (datoCorrectoFicha) {
    case 1:
        alfil2.setColor("n");
        alfil2.setPosicionX(posicionXCorrecta);
        alfil2.setPosicionY(posicionYCorrecta);
        return alfil2;
    case 2:
        caballo2.setColor("n");
        caballo2.setPosicionX(posicionXCorrecta);
        caballo2.setPosicionY(posicionYCorrecta);

```

```

        return caballo2;
    case 3:
        torre2.setColor("n");
        torre2.setPosicionX(posicionXCorrecta);
        torre2.setPosicionY(posicionYCorrecta);
        return torre2;
    default:
        break;
    }
    return null;
}

```

```

public static void juegoAvanzado(String[][] tablero){

    Scanner in = new Scanner(System.in);
    Ficha fichaBlanca = crearFichaBlancaAvanzada();
    boolean posicionInicialNoRepetida = false;
    Ficha fichaNegra = null;

    while(!posicionInicialNoRepetida){
        fichaNegra = crearFichaNegraAvanzada();
        if((fichaBlanca.getPosicionX()== fichaNegra.getPosicionX()) &&
(fichaBlanca.getPosicionY()== fichaNegra.getPosicionY())){
            System.out.println("La ficha negra no puede empezar en la misma posicion que
la blanca\nCrea de nuevo la ficha");
        } else {
            posicionInicialNoRepetida = true;
        }
    }

    tablero[fichaBlanca.getPosicionX()][fichaBlanca.getPosicionY()] =
fichaBlanca.getNombre()+fichaBlanca.getColor();
    tablero[fichaNegra.getPosicionX()][fichaNegra.getPosicionY()] =
fichaNegra.getNombre()+fichaNegra.getColor();
    boolean hayGanador = false;
    while(!hayGanador){
        hayGanador = ejecutarTurno(tablero, in, fichaBlanca, fichaNegra, "blancas");
        if (hayGanador) break;
        hayGanador = ejecutarTurno(tablero, in, fichaNegra, fichaBlanca, "negras");
    }

}

//Creamos para validar el movimiento dentro del juego avanzado
private static boolean ejecutarTurno(String[][] tablero, Scanner in, Ficha actual, Ficha rival,
String etiquetaTurno){
    boolean finTurno = false;

    do {

```

```

for(int i = 0; i<5; i++){
    System.out.println("\n");
    for(int j = 0; j<tablero[i].length;j++){
        System.out.print("\t"+tablero[i][j]);
    }
}
int prevX = actual.getPosicionX();
int prevY = actual.getPosicionY();
tablero[prevX][prevY] = "0";
Casilla destino = pedirDestino(in, etiquetaTurno);
if (actual.moverA(destino)) {
    actual.setPosicionX(destino.getFila());
    actual.setPosicionY(destino.getColumna());
    if (actual.getPosicionX() == rival.getPosicionX() && actual.getPosicionY() ==
rival.getPosicionY()){
        if (rival.getNombre() == 'A' && ((Alfil) rival).getVidasExtra() > 0){
            Alfil alfilRival = (Alfil) rival;
            alfilRival.setVidasExtra(alfilRival.getVidasExtra() - 1);
            System.out.println("¡El alfil del rival resiste! Vidas restantes: " +
alfilRival.getVidasExtra());
            tablero[rival.getPosicionX()][rival.getPosicionY()] = rival.getNombre() +
rival.getColor();
            actual.setPosicionX(prevX);
            actual.setPosicionY(prevY);
            tablero[prevX][prevY] = actual.getNombre() + actual.getColor();
        } else {
            tablero[actual.getPosicionX()][actual.getPosicionY()] = actual.getNombre() +
actual.getColor();
            for(int i = 0; i<5; i++){
                System.out.println("\n");
                for(int j = 0; j<tablero[i].length;j++){
                    System.out.print("\t"+tablero[i][j]);
                }
            }
            System.out.println("\nHan ganado las " + etiquetaTurno);
            return true;
        }
    } else {
        tablero[actual.getPosicionX()][actual.getPosicionY()] = actual.getNombre() +
actual.getColor();
    }
    if (actual.getNombre() == 'T' && ((Torre) actual).hasTurnosExtra()){
        Torre t = (Torre) actual;
        System.out.println("La torre tiene " + t.getTurnosExtra() + " turno(s) extra.
¿Quieres usar uno ahora? (s/n)");
        String resp = in.nextLine().trim().toLowerCase();
        if (resp.startsWith("s")){
            t.consumeTurnoExtra();

```

```

        finTurno = false;
    } else {
        finTurno = true;
    }
    } else {
        finTurno = true;
    }
    } else {
        System.out.println("Movimiento incorrecto");
        tablero[prevX][prevY] = actual.getNombre() + actual.getColor();
    }
} while (!finTurno);

return false;
}

private static Casilla pedirDestino(Scanner in, String etiquetaTurno){
    System.out.println("\nMueve la ficha " + etiquetaTurno + "\nElije el eje X");
    String movimiento = in.nextLine();
    int x = valorCorrecto(movimiento);
    System.out.println("Elije el eje Y");
    movimiento = in.nextLine();
    int y = valorCorrecto(movimiento);
    return new Casilla(x, y);
}

public static Ficha crearFichaBlancaAvanzada(){
    Scanner in = new Scanner(System.in);
    Alfil alfil1 = new Alfil();
    Ficha caballo1 = new Caballo();
    Torre torre1 = new Torre();
    boolean datoCorrecto = false;
    String eleccionFicha,posicionX,posicionY;
    int datoCorrectoFicha = 0;
    int posicionXCorrecta = -1;
    int posicionYCorrecta = -1;
    while(!datoCorrecto){
        System.out.println("Elije la primera ficha Blanca:\n1: Alfil\n2. Caballo\n3. Torre");
        eleccionFicha = in.nextLine();
        try{
            datoCorrectoFicha = Integer.parseInt(eleccionFicha);
            if(datoCorrectoFicha >=1 && datoCorrectoFicha<=3){
                datoCorrecto = true;
            }else{
                System.out.println("Debes de introducir un número entre 1 y 3");
            }
        }catch(NumberFormatException e){

```

```

        System.out.println("Debes de introducir un valor numérico sin decimales");
    }
}
datoCorrecto = false;
System.out.println("Elije una posicion de inicio X e Y");
while(!datoCorrecto){
    System.out.println("Posicion X (de 0 a 4):");
    posicionX = in.nextLine();
    try{
        posicionXCorrecta = Integer.parseInt(posicionX);
        if(posicionXCorrecta >= 0 && posicionXCorrecta <= 4){
            datoCorrecto = true;
        }else{
            System.out.println("Debes de introducir un número entre 0 y 4");
        }
    }catch(NumberFormatException e){
        System.out.println("Debes de introducir un valor numérico sin decimales");
    }
}
datoCorrecto = false;
while(!datoCorrecto){
    System.out.println("posicion Y(de 0 a 4):");
    posicionY = in.nextLine();
    try{
        posicionYCorrecta = Integer.parseInt(posicionY);
        if(posicionYCorrecta >= 0 && posicionYCorrecta <= 4){
            datoCorrecto = true;
        }else{
            System.out.println("Debes de introducir un número entre 0 y 4");
        }
    }catch(NumberFormatException e){
        System.out.println("Debes de introducir un valor numérico sin decimales");
    }
}
datoCorrecto = false;
switch (datoCorrectoFicha) {
    case 1:
        String vidas;
        int vidasNum = 0;
        alfil1.setColor("b");
        alfil1.setPosicionX(posicionXCorrecta);
        alfil1.setPosicionY(posicionYCorrecta);
        System.out.println("¿Cuántas vidas quieres que tenga el Alfil?");
        while(!datoCorrecto){
            vidas = in.nextLine();
            try{
                vidasNum = Integer.parseInt(vidas);
                datoCorrecto = true;
            }

```

```

        }catch(NumberFormatException e){
            System.out.println("Debes de introducir un valor numérico sin
decimales\n¿Cuántas vidas quieres que tenga el Alfil?");
        }
    }
    alfil1.setVidasExtra(vidasNum);
    return alfil1;
case 2:
    caballo1.setColor("b");
    caballo1.setPosicionX(posicionXCorrecta);
    caballo1.setPosicionY(posicionYCorrecta);
    return caballo1;
case 3:
    String turnos;
    int turnosNum = 0;
    torre1.setColor("b");
    torre1.setPosicionX(posicionXCorrecta);
    torre1.setPosicionY(posicionYCorrecta);
    System.out.println("¿Cuántos turnos extra quieres que tenga la torre?");
    while(!datoCorrecto){
        turnos = in.nextLine();
        try{
            turnosNum = Integer.parseInt(turnos);
            datoCorrecto = true;
        }catch(NumberFormatException e){
            System.out.println("Debes de introducir un valor numérico sin
decimales\n¿Cuántos turnos extra quieres que tenga la torre?");
        }
    }
    torre1.setTurnosExtra(turnosNum);
    return torre1;
default:
    break;
}
return null;
}

```

```

public static Ficha crearFichaNegraAvanzada(){
    Scanner in = new Scanner(System.in);
    Alfil alfil2 = new Alfil();
    Ficha caballo2 = new Caballo();
    Torre torre2 = new Torre();
    boolean datoCorrecto = false;
    String eleccionFicha,posicionX,posicionY;
    int datoCorrectoFicha = 0;
    int posicionXCorrecta = -1;
    int posicionYCorrecta = -1;
    while(!datoCorrecto){

```

```

System.out.println("Elige la primera ficha Negra:\n1: Alfín\n2. Caballo\n3. Torre");
    eleccionFicha = in.nextLine();
try{
    datoCorrectoFicha = Integer.parseInt(eleccionFicha);
    if(datoCorrectoFicha >=1 && datoCorrectoFicha<=3){
        datoCorrecto = true;
    }else{
        System.out.println("Debes de introducir un número entre 1 y 3");
    }
}catch(NumberFormatException e){
    System.out.println("Debes de introducir un valor numérico sin decimales");
}
}
datoCorrecto = false;
System.out.println("Elige una posicion de inicio X e Y");
while(!datoCorrecto){
    System.out.println("Posicion X (de 0 a 4):");
    posicionX = in.nextLine();
    try{
        posicionXCorrecta = Integer.parseInt(posicionX);
        if(posicionXCorrecta >= 0 && posicionXCorrecta <= 4){
            datoCorrecto = true;
        }else{
            System.out.println("Debes de introducir un número entre 0 y 4");
        }
    }catch(NumberFormatException e){
        System.out.println("Debes de introducir un valor numérico sin decimales");
    }
}
datoCorrecto = false;
while(!datoCorrecto){
    System.out.println("posicion Y(de 0 a 4):");
    posicionY = in.nextLine();
    try{
        posicionYCorrecta = Integer.parseInt(posicionY);
        if(posicionYCorrecta >= 0 && posicionYCorrecta <= 4){
            datoCorrecto = true;
        }else{
            System.out.println("Debes de introducir un número entre 0 y 4");
        }
    }catch(NumberFormatException e){
        System.out.println("Debes de introducir un valor numérico sin decimales");
    }
}
datoCorrecto = false;
switch (datoCorrectoFicha) {
    case 1:
        String vidas;

```

```

        int vidasNum = 0;
        alfil2.setColor("n");
        alfil2.setPosicionX(posicionXCorrecta);
        alfil2.setPosicionY(posicionYCorrecta);
        System.out.println("¿Cuántas vidas quieres que tenga el Alfil?");
        while(!datoCorrecto){
            vidas = in.nextLine();
            try{
                vidasNum = Integer.parseInt(vidas);
                datoCorrecto = true;
            }catch(NumberFormatException e){
                System.out.println("Debes de introducir un valor numérico sin
decimales\n¿Cuántas vidas quieres que tenga el Alfil?");
            }
        }
        alfil2.setVidasExtra(vidasNum);
        return alfil2;
    case 2:
        caballo2.setColor("n");
        caballo2.setPosicionX(posicionXCorrecta);
        caballo2.setPosicionY(posicionYCorrecta);
        return caballo2;
    case 3:
        String turnos;
        int turnosNum = 0;
        torre2.setColor("n");
        torre2.setPosicionX(posicionXCorrecta);
        torre2.setPosicionY(posicionYCorrecta);
        System.out.println("¿Cuántos turnos extra quieres que tenga la torre?");
        while(!datoCorrecto){
            turnos = in.nextLine();
            try{
                turnosNum = Integer.parseInt(turnos);
                datoCorrecto = true;
            }catch(NumberFormatException e){
                System.out.println("Debes de introducir un valor numérico sin
decimales\n¿Cuántos turnos extra quieres que tenga la torre?");
            }
        }
        torre2.setTurnosExtra(turnosNum);
        return torre2;
    default:
        break;
    }
    return null;
}
}

```

Casilla

```
public class Casilla {  
    private int fila = 5, columna = 5;  
  
    public Casilla() {  
    }  
    public Casilla(int fila, int columna){  
        this.fila = fila;  
        this.columna = columna;  
    }  
  
    public int getFila(){  
        return fila;  
    }  
  
    public int getColumna(){  
        return columna;  
    }  
}
```

Ficha

```
public abstract class Ficha {  
    private int posicionX, posicionY;  
    private String color;  
  
    public Ficha(){  
    }  
    public Ficha(int posicionX,int posicionY, String color){  
        this.posicionX = posicionX;  
        this.posicionY = posicionY;  
        this.color = color;  
    }  
  
    public int getPosicionX(){  
        return posicionX;  
    }  
    public void setPosicionX(int posicionX){  
        this.posicionX = posicionX;  
    }  
  
    public int getPosicionY(){  
        return posicionY;  
    }  
    public void setPosicionY(int posicionY){  
        this.posicionY = posicionY;  
    }  
}
```

```

    }

    public String getColor(){
        return color;
    }
    public void setColor(String color){
        this.color = color;
    }

    public abstract boolean moverA(Casilla casilla);
    public abstract char getNombre();
}

```

Alfil

```

public class Alfil extends Ficha{
    private final char nombre = 'A';
    private int vidasExtra;

    public Alfil(){
    }

    public Alfil(int posicionX, int posicionY, String color, int vidasExtra){
        super(posicionX,posicionY,color);
        this.vidasExtra = vidasExtra;
    }

    public int getVidasExtra(){
        return vidasExtra;
    }
    public void setVidasExtra(int vidasExtra){
        this.vidasExtra = vidasExtra;
    }

    @Override
    public char getNombre(){
        return nombre;
    }

    @Override
    public boolean moverA(Casilla casilla) {
        int xActual = getPosicionX();
        int yActual = getPosicionY();
        int xDestino = casilla.getFila();
        int yDestino = casilla.getColumna();
    }
}

```

```

        if(xActual == xDestino && yActual == yDestino ){
            return false;
        }

        int dx = Math.abs(xDestino - xActual);
        int dy = Math.abs(yDestino - yActual);
        return dx == dy;
    }
}

```

Caballo

```

public class Caballo extends Ficha {
    private final char nombre = 'C';

    public Caballo(){
    }

    public Caballo(int posicionX, int posicionY, String color){
        super(posicionX,posicionY,color);
    }

    @Override
    public char getNombre(){
        return nombre;
    }

    @Override
    public boolean moverA(Casilla casilla) {

        int xActual = getPosicionX();
        int yActual = getPosicionY();
        int xDestino = casilla.getFila();
        int yDestino = casilla.getColumna();
        if (xActual == xDestino && yActual == yDestino){
            return false;
        }
        int dx = Math.abs(xDestino - xActual);
        int dy = Math.abs(yDestino - yActual);
        return (dx == 2 && dy == 1) || (dx == 1 && dy == 2);
    }
}

```

Torre

```

public class Torre extends Ficha{

```

```

private char nombre = 'T';
private int turnosExtra;

public Torre(){
}

public Torre(int posicionX, int posicionY, String color, int turnosExtra){
    super(posicionX,posicionY,color);
    this.turnosExtra = turnosExtra;
}

public int getTurnosExtra(){
    return turnosExtra;
}

public void setTurnosExtra(int turnosExtra){
    this.turnosExtra = turnosExtra;
}

public boolean hasTurnosExtra(){
    return turnosExtra > 0;
}

public void consumeTurnoExtra(){
    if (turnosExtra > 0) turnosExtra--;
}

@Override
public char getNombre(){
    return nombre;
}

@Override
public boolean moverA(Casilla casilla) {
    int xActual = getPosicionX();
    int yActual = getPosicionY();
    int xDestino = casilla.getFila();
    int yDestino = casilla.getColumna();
    if (xActual == xDestino && yActual == yDestino){
        return false;
    }
    return (xActual == xDestino) || (yActual == yDestino);
}
}

```

Pruebas

```
--- exec:3.1.0:exec (default-cli) @ JuegoDeFichas ---  
Picked up JAVA_TOOL_OPTIONS: -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF8-8
```

```
0      0      0      0      0  
0      0      0      0      0  
0      0      0      0      0  
0      0      0      0      0  
0      0      0      0      0
```

JUEGO DE FICHAS

```
1. Normas  
2. Juego básico  
3. Juego Avanzado  
4. Exit  
3  
Elige la primera ficha Blanca:  
1: Alfil  
2. Caballo  
3. Torre  
1  
Elige una posicion de inicio X e Y  
Posicion X (de 0 a 4):  
0  
posicion Y(de 0 a 4):  
0  
¿Cuántas vidas quieres que tenga el Alfil?  
2  
Elige la primera ficha Negra:  
1: Alfiln  
2. Caballo  
3. Torre  
3  
Elige una posicion de inicio X e Y  
Posicion X (de 0 a 4):  
4  
posicion Y(de 0 a 4):  
4  
¿Cuántos turnos extra quieres que tenga la torre?  
2
```

Ab	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	Tn

Mueve la ficha blancas

Elije el eje X

1

Elije el eje Y

1

0	0	0	0	0
0	Ab	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	Tn

Mueve la ficha negras

Elije el eje X

4

Elije el eje Y

1

La torre tiene 2 turno(s) extra. ¿Quieres usar uno ahora? (s/n)

s

0	0	0	0	0
0	Ab	0	0	0
0	0	0	0	0
0	0	0	0	0
0	Tn	0	0	0

Mueve la ficha negras

Elije el eje X

1

Elije el eje Y

1

¡El alfil del rival resiste! Vidas restantes: 1
La torre tiene 1 turno(s) extra. ¿Quieres usar uno ahora? (s/n)
s

0	0	0	0	0
0	Ab	0	0	0
0	0	0	0	0
0	0	0	0	0
0	Tn	0	0	0

Mueve la ficha negras

Elige el eje X

1

Elige el eje Y

1

¡El alfil del rival resiste! Vidas restantes: 0

0	0	0	0	0
0	Ab	0	0	0
0	0	0	0	0
0	0	0	0	0
0	Tn	0	0	0

Mueve la ficha blancas

Elige el eje X

0

Elige el eje Y

0

Ab	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	Tn	0	0	0

Mueve la ficha negras

Elige el eje X

0

Elige el eje Y

Elige el eje Y

4

Movimiento incorrecto

Ab	0	0	0	0
----	---	---	---	---

0	0	0	0	0
---	---	---	---	---

0	0	0	0	0
---	---	---	---	---

0	0	0	0	0
---	---	---	---	---

0	Tn	0	0	0
---	----	---	---	---

Mueve la ficha negras

Elige el eje X

4

Elige el eje Y

0

Ab	0	0	0	0
----	---	---	---	---

0	0	0	0	0
---	---	---	---	---

0	0	0	0	0
---	---	---	---	---

0	0	0	0	0
---	---	---	---	---

Tn	0	0	0	0
----	---	---	---	---

Mueve la ficha blancas

Elige el eje X

1

Elige el eje Y

1

0	0	0	0	0
---	---	---	---	---

0	Ab	0	0	0
---	----	---	---	---

0	0	0	0	0
---	---	---	---	---

0	0	0	0	0
---	---	---	---	---

Tn	0	0	0	0
----	---	---	---	---

Mueve la ficha negras

Elige el eje X

0

U U U U U
 Mueve la ficha blancas
 Elije el eje X
 1
 Elije el eje Y
 1

0	0	0	0	0
0	Ab	Tn	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

Mueve la ficha negras
 Elije el eje X
 1
 Elije el eje Y
 1

0	0	0	0	0
0	Tn	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

Han ganado las negras

JUEGO DE FICHAS

1. Normas
2. Juego básico
3. Juego Avanzado
4. Exit